Module 2: Sorting and Order Statistics Algorithms

Lesson 1: O(n^2) Sorting Algorithms

Learning Outcomes

- LO 2.1.1 Analyze O(n^2) sorting algorithms

N^2 Sorting Algorithms

Putting an unsorted list of data elements into order—*sorting*—is a very common and useful operation. Entire books have been written about various sorting algorithms. The goal is to come up with better, more efficient sorts. Because sorting a large number of elements can be extremely time-consuming, a good sorting algorithm is very desirable. This is one area in which programmers are sometimes encouraged to sacrifice clarity in favor of speed of execution.

How do we describe efficiency? We pick an operation central to most sorting algorithms: the operation that compares two values to see which is smaller. In our study of sorting algorithms, we relate the number of comparisons to the number of elements in the list (N) as a rough measure of the efficiency of each algorithm. The number of swaps made is another measure of sorting efficiency.

Another efficiency consideration is the amount of memory space required. In general, memory space is not a very important factor in choosing a sorting algorithm. We look at only one sort in which space would be a serious consideration. The usual time versus space tradeoff applies to sorts—more space often means less time, and vice versa.

Because processing time is the factor that applies most often to sorting algorithms, we consider it in detail here. Of course, as in any application, the programmer must determine goals and requirements before selecting an algorithm and starting to write code

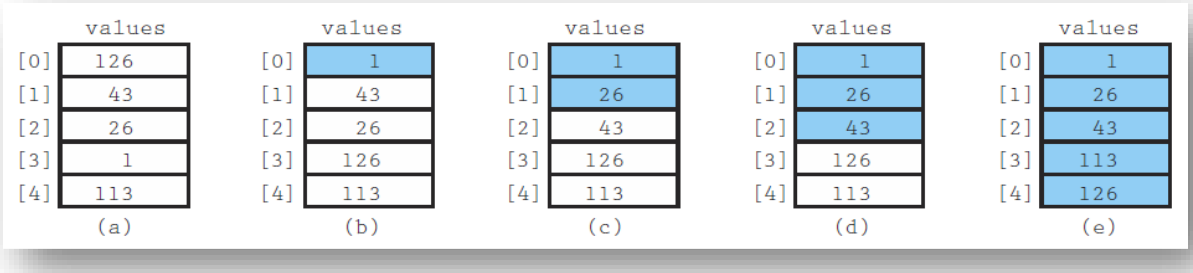
Selection Sort

If you were handed a list of names and asked to put them in alphabetical order, you might use this general approach:
Find the name that comes first in the alphabet, and write it on a second sheet of paper.

- Cross the name out on the original list.
- Continue this cycle until all names on the original list have been crossed out and written onto the second list, at which point the second list is sorted.

This algorithm is simple to translate into a computer program, but it has one drawback:
It requires space in memory to store two complete lists. Although we have not talked a great deal about memory space considerations, this duplication is clearly wasteful.

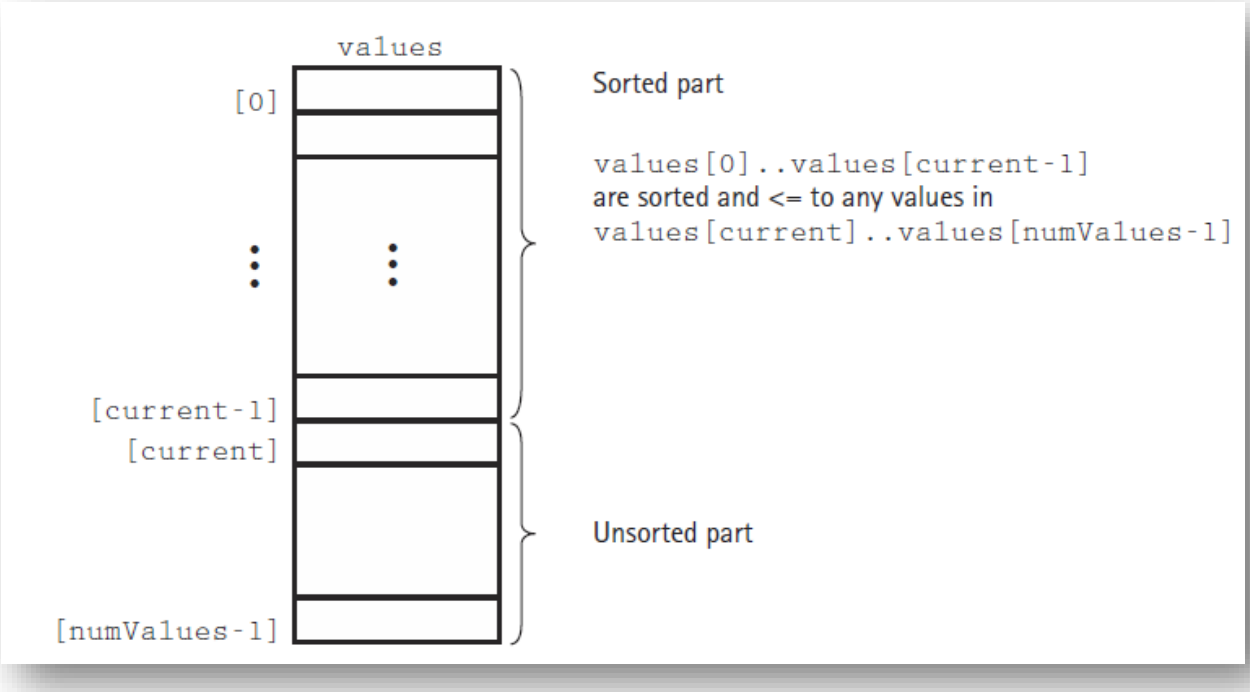
A slight adjustment to this manual approach does away with the need to duplicate space, however. As you cross a name off the original list, a free space opens up. Instead of writing the minimum value on a second list, you can exchange it with the value currently in the position where the crossed-off item should go. Our “by-hand list” is represented in an array.



Selection Sort Pseudo-code

Set current to the index of first item in the array

```
while more items in unsorted part of array
    Find the index of the smallest unsorted item
    Swap the current item with the smallest unsorted one
    Shrink the unsorted part of the array by incrementing current
```



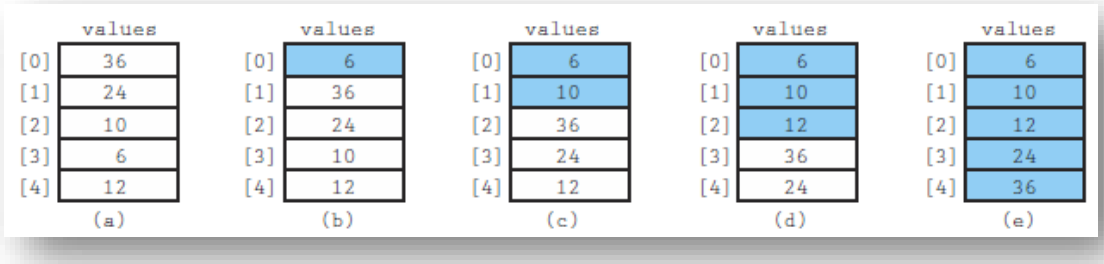
The straight selection sort requires *N*(*N* - 1)/2 comparisons. Note that the particular arrangement of values in the array does not affect the amount of work done. Even if the array is in sorted order *before* the call to **SelectionSort**, the function still makes *N*(*N* - 1)/2 comparisons. In Big-O notation, we consider only the term ½ (N^2,) because it increases fastest relative to N. Further, we ignore the constant, ½ making this algorithm become O (N^2). Thus, for large values of N, the computation time is approximately proportional to N^2.

Selection Sort C style implementation

```
void selectionSort()
{
    int a[5] = {126,43,26,1,113};
    int small,idx_small;

    for (int i=0; i<4;i++)
    {
        idx_small = i;
        for (int j=i+1;j<5;j++)
        {
            if (a[j]<a[idx_small])
            {
                idx_small = j;
            }
        }
    swap (a[i],a[idx_small]);
    }
    for (int idx=0;idx<5;idx++)
        cout<<a[idx]<<",";
}
```

Bubble Sort

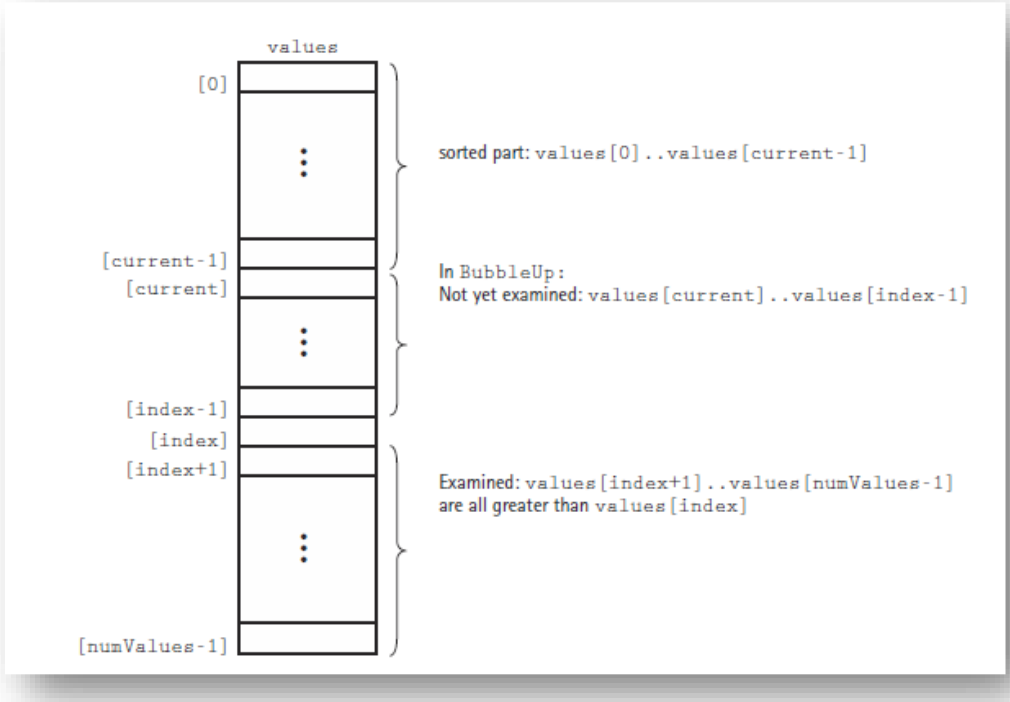
The bubble sort is a selection sort that uses a different scheme for finding the minimum (or maximum) value. Each iteration puts the smallest unsorted element into its correct place, but it also changes the locations of the other elements in the array. The first iteration puts the smallest element in the array into the first array position. Starting with the last array element we compare successive pairs of elements, swapping them whenever the bottom element of the pair is smaller than the one above it. In this way, the smallest element “bubbles up” to the top of the array. The next iteration puts the smallest element in the unsorted part of the array into the second array position, using the same technique.



Bubble Sort Pseudo-code

```
Set current to the index of first item in the array
while more items in unsorted part of array
      “Bubble up” the smallest item in the unsorted part,
```

```
causing intermediate swaps as needed
Shrink the unsorted part of the array by incrementing current
```



Bubble Sort C-style implementation

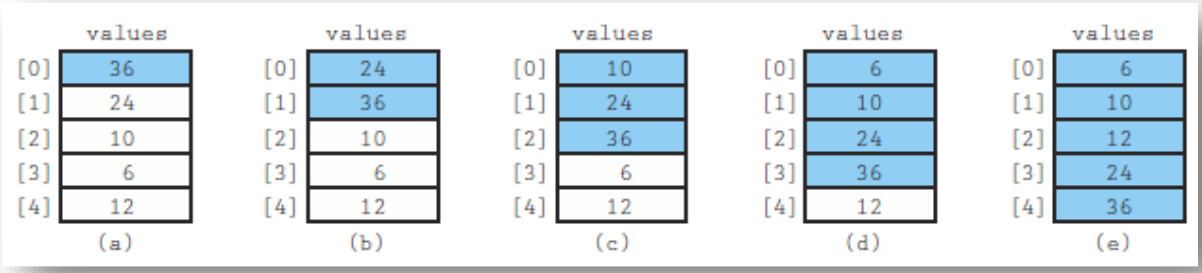
```
void bubbleSort()
{
    int a[5] = {36,24,10,6,12};
    int i=0;
    bool swp = 1;

    while (swp)
    {
        swp = 0;
        for (int j=4;j>i;j--)
        {
            if (a[j]<a[j-1])
            {
                swap (a[j],a[j-1]);
                swp = 1;
            }
        }
        i++;
    }
    for (int idx=0;idx<5;idx++)
        cout<<a[idx]<<",";
}
```

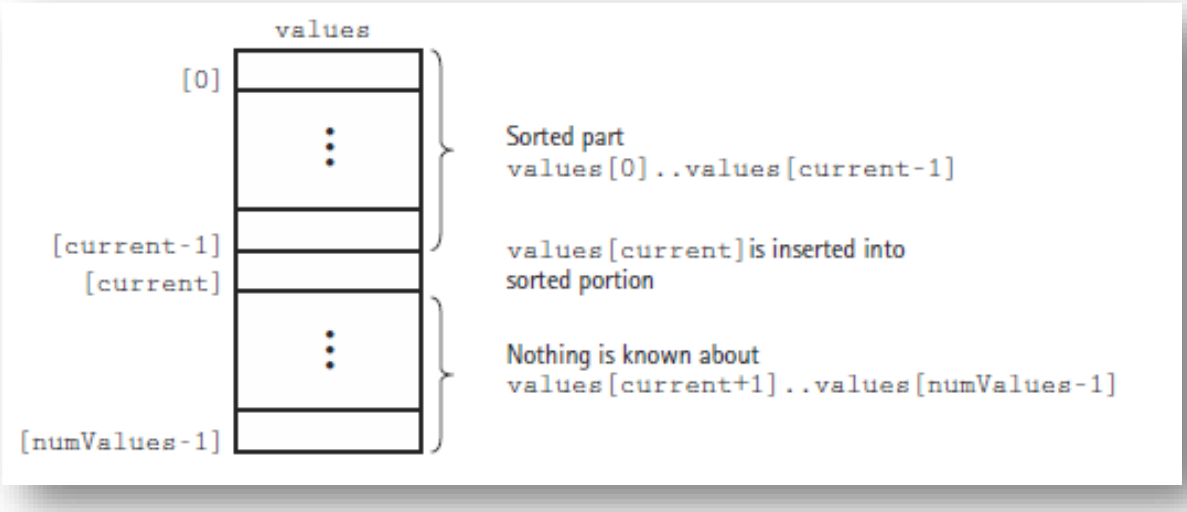
Clearly, if the array is initially sorted in order, only *N* - 1 comparisons are required for the sort. What if the original array was actually sorted in *descending* order? This is the worst possible case: requires as many comparisons as **BubbleSort** and **SelectionSort**, not to mention the “overhead” consisting of the many extra swaps and setting and resetting of the **sorted** flag. Can we calculate an average case? Analyzing the size of the array N and K bubble swaps we get.

$(N - 1) + (N - 2) + (N - 3) + \ldots + (N - K) = (2KN - K^2 - K) / 2$
In Big-O notation, the term that is increasing the fastest relative to N is $2KN$. We know that K is between 1 and N - 1. On average, over all possible input orders, K is proportional to N. Therefore, $2KN$ is proportional to N^2.

Insertion Sort

The principle of the insertion sort is quite simple: Each successive element in the array to be sorted is inserted into its proper place with respect to the other, already-sorted elements. As with the previous sorts, we divide our array into a sorted part and an unsorted part. Initially, the sorted portion contains only one element: the first element in the array. Now we take the second element in the array and put it into its correct place in the sorted part; that is, values[0] and values[1] are in order with respect to each other. Now the value in values[2] is put into its proper place, so values[0] . . values[2] are in order with respect to each other. This process continues until all the elements have been sorted.





Insertion sort Pseudo – code

```
for count going from 0 through numValues - 1
```

```
        InsertItem(values, 0, count)
```

Insertion sort C-style implementation

```
void insertionSort()
{
     int a[5] = {36,24,10,6,12};
     int key,j;
     for (int i=1;i<5;i++)
     {
        key=a[i];
        j=i-1;
        while(key<a[j]&&j>=0)
        {
            a[j+1]=a[j];
            j--;
        }
        a[j+1] = key;
        for (int idx=0;idx<5;idx++)
            cout<<a[idx]<<",";
        cout<<endl;
     }

}
```

The general case for this algorithm mirrors that for the **SelectionSort** and the **BubbleSort**, so the general case is O(N^2). Like **InsertionSort** also has a best case: the data are already sorted in ascending order. When the data are in ascending order, **InsertItem** is called *N* times, but only one comparison is made each time and no swaps occur. The maximum number of comparisons is made only when the elements in the array are in reverse order.

Shell Sort

The running time of insertion sort is quadratic because it interchanges elements which are side with each other. Shell sort, named after Donald Shell attempts to improve this by comparing and interchanging elements which are far apart from each other. The principle behind Shellsort is that at the beginning the elements which are far apart from each other are compared and interchanged. This distance is decreased until the last phase in which the adjacent elements are compared and interchanged if necessary.

C-style implementation of Shell sort

```
void shellSort()
{
     int a[5] = {15,24,10,6,12};

     int n=5,i,j,d;
     int tmp;
     for (d=n/2;d>0;d/=2)
     {
        for (i=d;i<n;i++)
        {   tmp = a[i];
           for (j=i;j>d-1;j-=d)
             {
                 if (tmp<a[j-d])
                    a[j]=a[j-d];
```

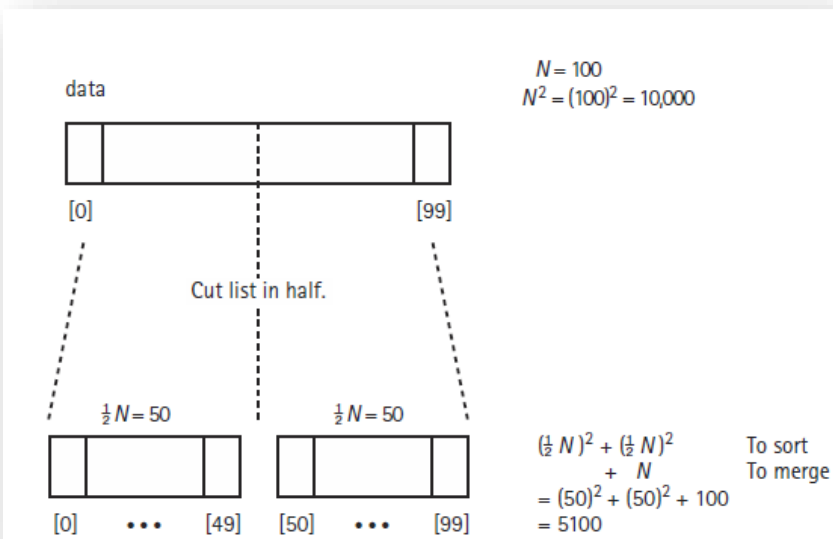
```
                    else
                       break;
                }
                a[j]=tmp;

        }

    }
      for (int idx=0;idx<5;idx++)
            cout<<a[idx]<<",";
           //cout<<endl;
}
```

Lesson 2: O(n log (n)) sorting algorithms and O(n) order statistics

Learning Outcomes

- LO 2.2.1 Analyze O(n logn) sorting algorithms
- LO 2.2.2 Analyze O(n) algorithm for median and order statistics
-

Considering how rapidly *N*2 grows as the size of the array increases, can't we do better? We note that *N*2 is a lot larger than $(\frac{1}{2} N)^2 + (\frac{1}{2} N)^2$. If we could cut the array into two pieces, sort each segment, and then merge the two pieces again, we should end up sorting the entire array with a lot less work. The figure below shows an example of this approach.



The idea of "divide and conquer" has been applied to the sorting problem in different ways, resulting in a number of algorithms that can do the job much more efficiently than $O(N^2)$. In fact, an entire category of sorting algorithms are $O(N \log_2 N)$.

Merge Sort

Merge Sort Pseudo - code

```
Cut the array in half
Sort the left half
Sort the right half
Merge the two sorted halves into one sorted array
```

Merging the two halves together is an O(N) task: We merely go through the sorted halves, comparing successive pairs of values (one in each half) and putting the smaller value into the next slot in the final solution. Even if the sorting algorithm used for each half is $O(N^2)$, we should see some improvement over sorting the whole array at once. That's right—we can make MergeSort be a recursive function and let it call itself to sort each of the two subarrays:

MergeSort—Recursive

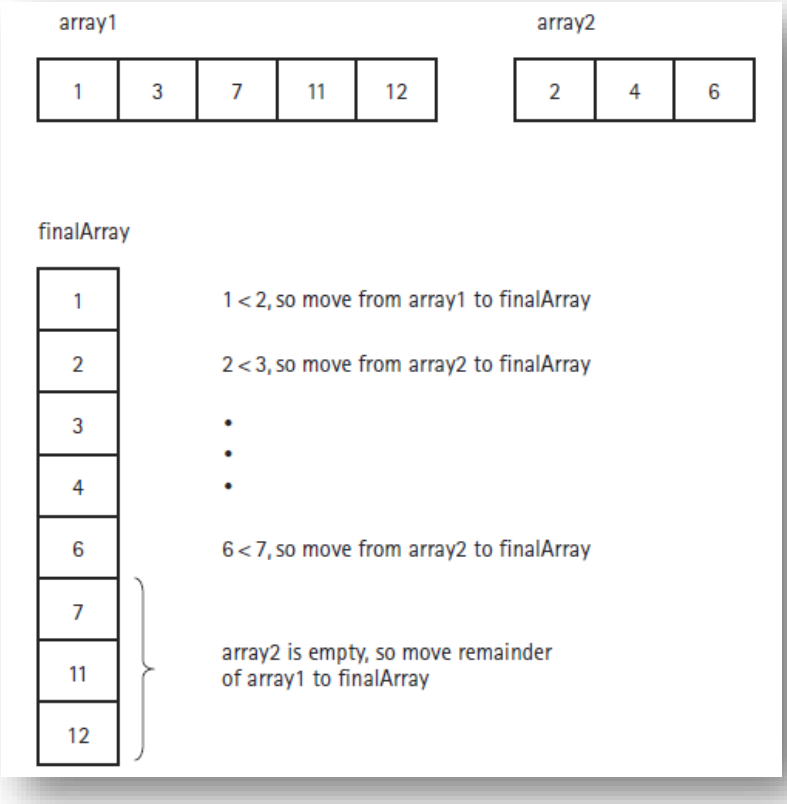
```
Cut the array in half
MergeSort the left half
MergeSort the right half
Merge the two sorted halves into one sorted array
```

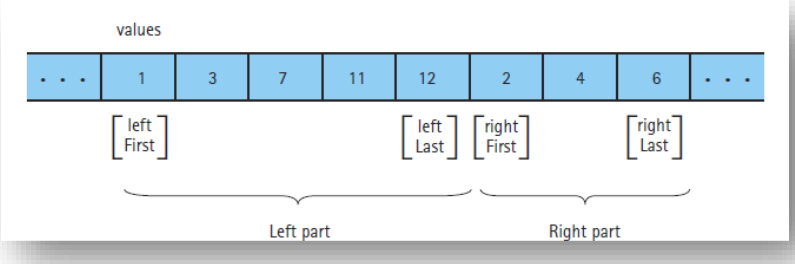
This is the general case, of course. What is the base case, which does not involve any recursive calls to **MergeSort**? If the "half " to be sorted doesn't have more than one element, we can consider it already sorted and just return.

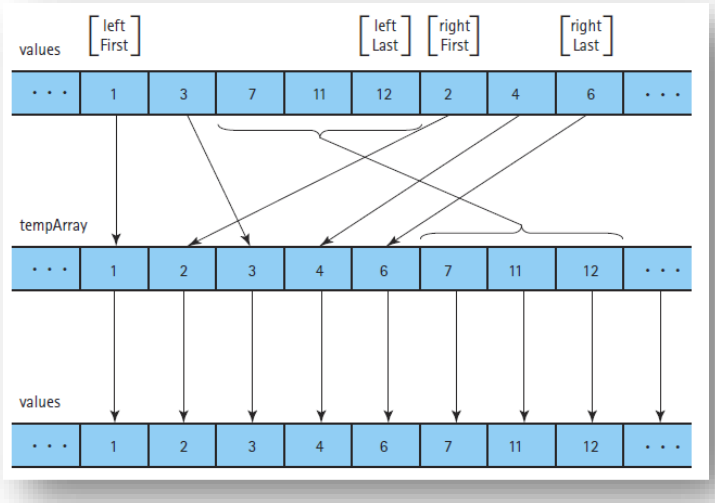
C-style implementation of Merge sort

```
void mergeSort(int a[],int start,int end)
{
    int mid;
    if(start<end)
    {
        mid = (start+end)/2;
        mergeSort(a,start,mid);

        mergeSort(a,mid+1,end);
        cout<<"start: "<<start<<endl;
        cout<<"mid: "<<mid+1<<endl;
        mergeArr(a,start,mid,end);
    }
}
```

Obviously, the serious work occurs in the merge step. Let's look first at the general algorithm for merging two sorted arrays, and then at the specific problem of our subarrays. To merge two sorted arrays, we compare successive pairs of elements, one from each array, moving the smaller of each pair to the "final" array. We can stop when the shorter array runs out of elements, and then move all remaining elements (if any) from the other array to the final array.


array1
1
3
7
11
12
array2
2
4
6
finalArray
1
2
3
4
6
7
11
12
1 < 2, so move from array1 to finalArray
2 < 3, so move from array2 to finalArray
6 < 7, so move from array2 to finalArray
array2 is empty, so move remainder of array1 to finalArray


values
. . .
1
3
7
11
12
2
4
6
. . .
left First
left Last
right First
right Last
Left part
Right part



C-style implementation of merging

```
void mergeArr(int a[],int start, int mid, int end)
{
    int temp[8],size,ptr1,ptr2;
    int i,j;

    size = end - start;

    ptr1=start;
    ptr2 =mid+1;

    j=0;
    /* The while loop below would execute  equal to the
     minimum of mid and (n-mid)
    */
    while(ptr1<mid+1&&ptr2<end+1)
    {
        if (a[ptr1]<a[ptr2])
        {
            temp[j]=a[ptr1];
            ptr1++;
            j++;
        }
        else
        {
            temp[j]=a[ptr2];
            ptr2++;
            j++;
        }
    }

    if(ptr1<mid+1)
    {
        for (i=j;i<size+1;i++)
        {
                temp[i]=a[ptr1];
                ptr1++;
```

```
        }
    }

    if(ptr2<end+1)
    {
            for (i=j;i<size+1;i++)
            {
                temp[i]=a[ptr2];
                ptr2++;
            }
    }

        for (j=0;j<size+1;j++)
            a[start+j]=temp[j];
}
```

Quick Sort

The analysis of QuickSort is very similar to that of MergeSort. On the first call, every element in the array is compared to the dividing value (the “split value”), so the work done is O(*N*). The array is divided into two parts (not necessarily halves) using a pivot, which are then examined. Each of these pieces is then divided in two, and so on. If each piece is split approximately in half, $O(\log_2 N)$ splits occur. At each split, we make O(N) comparisons. Thus QuickSort is also an $O(N\log_2 N)$ algorithm, which is quicker than the $O(N^2)$ sorts.

Quick Sort Pseudo – code

```
if there is more than one item in values[first]..values[last]
      Select splitVal
      Split the array so that
values[first]..values[splitPoint-1] <= splitVal values[splitPoint]
= splitVal
      values[splitPoint+1]..values[last] > splitVal
QuickSort the left half
QuickSort the right half
```

C-style implementation of Quick sort

```
void quickSort(int a[],int start, int end)
{
    int i, j, pivot;
    if (start<end)
    {
        pivot = a[start];
        i=start+1;
        while(a[i]<pivot)
            i++;
        j=end;
        while(a[j]>pivot)
            j--;
        while(i<j)
        {
            swap(a[i],a[j]);
            i++;
            while(a[i]<pivot)
                i++;
            j--;
            while(a[j]>pivot)
                j--;
        }
     swap(a[start],a[j]);
    cout<<endl;
    quickSort(a,start,j-1);
    quickSort(a,j+1,end);
    }
}
```

Order statistics

The ith order statistic of a set of n elements is the ith smallest element. The selection problem can be specified as follows:

- Input: A set A of n (distinct) elements and a number 1, with $1 \le i \le n$
- Output: The element $x \in A$ that is larger than i-1 other elements of A

The selection problem can be solved in $O(n^2)$ using partial selection sort, or O (n log n) by sorting the entire array using merge sort or heap sort. However by taking advantage of the pivoting of quick sort, a linear time algorithm is applicable for the problem.

Quickselect C-style implementation

```
int quickSelect(int a[],int start ,int end, int IthIdx)
{
    int q,k,i;
    if (start==end)
        return (a[start]);
    q = Partition(a,start,end);
    k= q - start+1;
   if (IthIdx<k)
        return (quickSelect(a,start,q, IthIdx));
   else
       return (quickSelect(a,q+1,end,IthIdx-k));
}
```

The partition operation is similar to quicksort.

Partition C-style implementation

```
int Partition(int a[],int start, int end)
{
    int pivot = a[start];
    int i = start + 1;

    while (a[i]<pivot)
        i++;
    int j=end;

    while (a[j]>pivot)
        j--;

    while (i<j)
    {
        swap (a[i],a[j]);
        i++;
        while (a[i]<pivot)
            i++;
        j--;
        while (a[j]>pivot)
            j--;
    }
    swap (a[start],a[j]);
    return j;

}
```